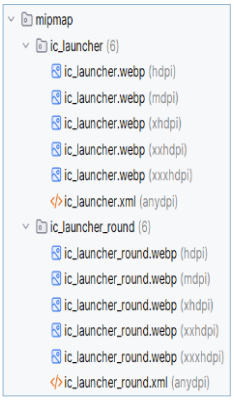
Ressources – La classe R

28

La classe mipmap (icônes de l'application)

- Le dossier **mipmap** est utilisé surtout pour les icônes de l'application (ex. ic_launcher).
- Android crée **un seul identifiant** pour un fichier qui a le même nom, même si tu as plusieurs versions de ce fichier pour différentes résolutions d'écran (mipmap-mdpi, mipmap-hdpi, etc.).

```
public static final class mipmap {
  public static final int ic_launcher=0x7f0a0000;
  public static final int ic_launcher_round=0x7f0a0001;
}
```



- Quand on appelle R.mipmap.ic_launcher, Android choisit **automatiquement la bonne version** adaptée à l'écran du téléphone.
- De même, l'identifiant d'un mipmap est accessible comme suit:

Java / Kotlin

```
R.mipmap.ic_launcher
```

XML

```
"@mipmap/ic_launcher"
```

28

Ressources – La classe R

29

La classe String

- Cette fois, pour toute chaîne de caractères dans le fichier ***res/values/strings.xml*** correspond un entier dans la classe string.

```
public static final class string {
  public static final int app_name=0x7f0b0027;
  public static final int course=0x7f0b0028;
  public static final int hello_world=0x7f0b0029;
}
```

- Chaque texte défini dans ce fichier reçoit un identifiant dans **R.string**.
- L'identifiant d'une chaine de caractères est accessible comme suit:

Java / Kotlin

```
R.string.hello_world
```

XML

```
"@string/hello_world"
```

➔ **Avantage** : séparer les textes du code et de faciliter la traduction.

- Cette externalisation permet l'internationalisation de l'application (traductions multi-langues).

29

Ressources – La classe R

30

***Autres valeurs simples* (res/values)**

- Dans le dossier **res/values**, on peut mettre plusieurs fichiers XML :
  - colors.xml → définit les couleurs.
  - dimens.xml → définit des dimensions (tailles).
  - arrays.xml → définit des tableaux de valeurs.
- Tous ces fichiers génèrent aussi des identifiants dans la classe R.



30

Ressources – La classe R

31

- Autres répertoires de ressources pris en charge dans le répertoire res/ du projet.

Répertoire de ressource	Type de ressource
drawable-ldpi	Images en basse définition 240 × 320 pixels
drawable-mdpi	Images en moyenne définition 320 × 480 pixels
drawable-hdpi	Images en haute définition 480 × 800 pixels
drawable-xhdpi	Images en extra haute définition 720×1280 pixels
anim	Fichiers XML qui définissent les animations.
color	Fichiers XML qui définissent une liste d'état des couleurs.
menu	Les fichiers XML qui définissent les menus de l'application.
raw	Fichiers arbitraires à sauvegarder dans leur forme brute.
xml	Fichiers XML arbitraires qui peuvent être lus au moment de l'exécution.

31

Internationalisation

32

- Android utilise **des ressources localisées** pour gérer l'internationalisation et la traduction de chaînes de caractères.
- Android gère plusieurs langues en créant des sous-dossiers dans res/values/.
  - Il suffit de créer des répertoires values-XX où XX est le code de la langue que l'on souhaite implanter.
  - On place alors dans ce sous répertoire le fichier xml strings.xml contenant les chaines traduites associées aux mêmes clefs que dans values/strings.xml.
- La convention veut que le dossier contienne la langue associée. Pour chaque langue, on crée un sous-dossier spécifique :
  - res\values-fr pour le français,
  - res\values-en pour l'anglais et ainsi de suite.
  - res\values-es pour l'espagnol
  - res\values-fr-rFR pour le français de France.
  - res\values-en-rUS pour l'anglais des états-Unis
  - Etc
- → Android choisit automatiquement la bonne langue selon la configuration du téléphone.

32

Emploi des ressources texte

33

- Dans une Activity, pour récupérer une chaîne définie dans res/values/strings.xml et l'utiliser dans le programme, on utilise :

```
val msg = getString(R.string.message)
```

  - getString() est une méthode de la classe Activity (héritée de la classe Context).
  - Elle permet d'accéder directement à une ressource de type chaîne en utilisant son identifiant (R.string.nom_chaine).
- En dehors d'une Activity (par exemple dans une classe utilitaire), on passe par un contexte :

```
val msg = context.getString(R.string.message)
```

- → Android gère automatiquement la langue affichée selon la configuration de l'appareil.

33

Les couleurs

34

- Elles se placent dans res/values/colors.xml :
  - Chaque couleur est décrite par son **nom** et son **code RGB hexadécimal**.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

- Dans le fichier ressources xml :
  - **android:textColor="@color/purple_200"**
  - **android:background="@color/black"**
- Dans le code Kotlin :

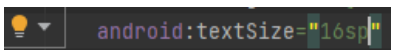
  // Depuis une Activity ou Fragment
  - **val color = ContextCompat.getColor(this, R.color.purple_200)**
  - **textView.setTextColor(color)**

34

Les dimensions

35

- Les tailles (marges, polices, etc.) sont définies dans **dimens.xml**.
- Une dimension est spécifiée par ***un nombre suivi d'une unité de mesure.***
  - Par exemple : 10px, 16dp, 5sp.



- Les unités de mesure suivantes sont prises en charge par Android :
  - **dp:** pour assurer la cohérence des tailles réelles de vos éléments d'interface utilisateur sur différents appareils.
    - L'avantage de cette mesure est que le système applique un ratio de correction en fonction des capacités réelles de l'écran pour assurer un affichage correct.
  - **sp:** pour spécifier la taille des polices, afin qu'elle soit ajustée à la densité de l'écran.
  - **px:** pour pixels réels sur l'écran. Cette unité de mesure n'est pas recommandée car la représentation réelle peut varier d'un appareil à l'autre.

35

Les dimensions

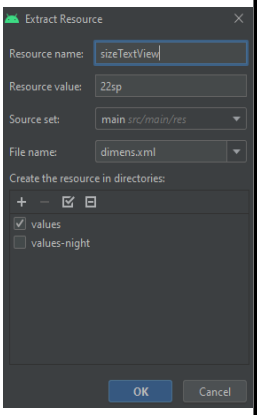
36

L'identification d'une dimension est accessible comme suit :

- Dans le fichier ressources xml :
  - **android:textSize="@dimen/sizeTextView "**
- Dans le code Kotlin :

// Depuis une Activity ou Fragment

  - **val dimen = resources.getDimension(R.dimen.sizeTextView)**
  - **textView.textSize = dimen**



36

Les images

37

- Ajouter une image dans une application Android est très simple :

1. Placer l'image dans le répertoire approprié du projet :
   - res/drawable/ → pour les images classiques (PNG, JPG, XML vector drawables).
   - res/mipmap/ → pour les icônes de l'application (ex. ic_launcher).
2. Android Studio permet de glisser-déposer directement les fichiers dans ces dossiers.
3. Les ressources sont accessibles via l'identifiant généré dans la classe **R.**

- Dans le fichier xml :
  - **android:src="@drawable/mon_image"**
- Dans le code Kotlin : // Exemple dans une Activity ou Fragment
  - **imageView.setImageResource(R.drawable.mon_image)**

37

Les animations

38

- Une **animation** est le processus qui ajoute un effet de mouvement ou de transformation à une vue (texte, image, bouton…).
- Elle rend l'interface utilisateur plus dynamique et attractive.

Types d'animations en Android

1. **Animations de propriété (Property Animation API)**
   - Permettent de modifier progressivement les propriétés d'une vue (ex. translation, rotation, opacité).
   - **Classe principale** : ObjectAnimator.
   - **Exemple en Kotlin** :

```
val animator = ObjectAnimator.ofFloat(imageView, "rotation", 0f, 360f)
animator.duration = 1000
animator.start()
```

38

Les animations

39

Types d'animations en Android

2. **View Animation (Tween Animation, res/anim/)**
   - Définies en XML dans le dossier res/anim/.
   - **Exemple** : res/anim/fade_in.xml
   - **Utilisation en Kotlin** :

```
val anim = AnimationUtils.loadAnimation(this, R.anim.fade_in)
imageView.startAnimation(anim)
```

3. **Drawable Animation (Animation de ressources dessinables)**
   - Suite d'images affichées successivement (animation frame-by-frame).
   - Définies en XML dans res/drawable/.

39

Ressources – Organisation

40

- **Principe général :** Les ressources Android (**images, layouts, textes, etc.**) peuvent s'adapter **au matériel de l'utilisateur** (taille écran, langue, orientation…).
- **Par défaut** : Android va chercher dans les dossiers standards (res/drawable, res/layout, res/values, …).
- **Mais** on peut créer des variantes (ex. res/layout-fr, res/drawable-xhdpi) pour que l'application choisisse la ressource **la mieux adaptée.**
  - Et si Android ne trouve pas la ressource exacte ?
    - il va chercher l'option la plus proche dans les autres dossiers disponibles.

40

Ressources – Organisation

41

- En partant du nom du répertoire par défaut, on peut créer d'autres dossiers qui permettent de préciser à quels types de matériels les ressources de ce répertoire sont destinées.
  - Il y a une syntaxe très précise à respecter pour qu'Android sache dans quel répertoire il doit fouiller.
  - Les restrictions sont représentées par des qualificateurs ((langue, orientation, taille écran, densité, version Android, …), au nombre de 20, qui permettront de préciser le matériel de destination.

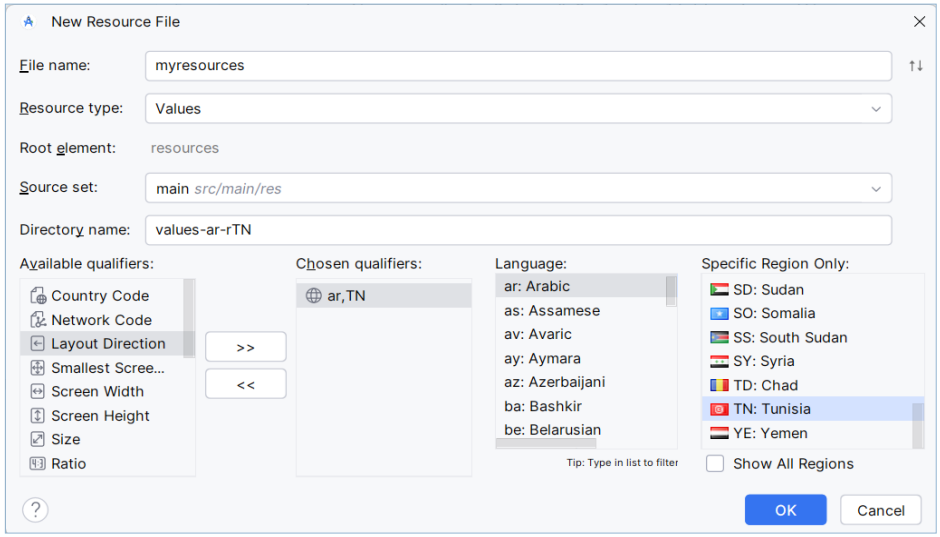
res/<type_ressource>[<-qualificateur1><-qualificateur 2>…<-qualificateur N>]

  - Ces qualificateurs doivent être ordonnés selon leurs priorités : Android lit les qualificateurs **par priorité** (langue, puis taille, puis orientation, etc.).

41

Ressources – Organisation

42



42

Ressources – Organisation

Quelques qualificateurs

43

- **Langue et région (Locale : Priorité 3)**
  - Permet d'adapter les ressources à la langue préférée de l'utilisateur.
  - On indique une langue puis, éventuellement, on peut préciser une région avec « -r ».
  - Exemples :
    - « en » pour l'anglais ;
    - « fr » pour le français (tous les pays) ;
    - « fr-rFR » pour le français mais uniquement celui utilisé en France ;
    - « fr-rCA » pour le français mais uniquement celui utilisé au Québec ;
    - etc.

43

Ressources – Organisation

Quelques qualificateurs

44

- **Taille de l'écran (Size : Priorité 8)**
  - Dépend de la diagonale de l'écran :
    - small
    - normal
    - large
    - xlarge
- **Orientation de l'écran (Orientation : Priorité 10)**
  - port : Portrait (par défaut) → res/layout/activity_main.xml
  - land : Landscape (paysage). → res/layout-land/activity_main.xml

44

Ressources – Organisation

Quelques qualificateurs

45

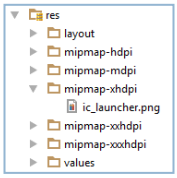
- **Résolution de l'écran (Density : Priorité 13)**
  - Dépend de la **résolution (dpi)** de l'écran :
    - Low density (ldpi) : environ 120 dpi
    - Medium density (mdpi) : environ 160 dpi
    - High density(hdpi) : environ 240 dpi
    - X-High density (xhdpi) : environ 320 dpi
    - XX-High density (xxhdpi) : environ 480 dpi
    - No density (nodpi) : pour ne pas redimensionner les images matricielles.
- **Version d'Android (Version : Priorité 20)**
  - Permet de cibler une API minimale (v19, v20, v21, v25, etc.).
  - Utile si une fonctionnalité ou mise en page existe seulement à partir d'une version d'Android.

45

Ressources – Organisation

Quelques qualificateurs

46



- Exemples
  - **res/mipmap-xhdpi** pour avoir des images spécifiquement pour les écrans à **très haute densité.**
  - **res/layout-fr-rFR** pour avoir une mise en page spécifique destinée à ceux qui ont choisi la langue « **Française de France** ».
  - **res/values-fr-rCA-port** pour des données qui s'afficheront uniquement à ceux qui ont choisi la langue « **Française du Québec** » et dont le téléphone se trouve en orientation **portrait.**

Une mise en page destinée à la version 25, quand l'utilisateur penche son téléphone en mode portrait alors qu'il utilise le Français des Belges et que son terminal a une résolution moyenne.

res/layout-fr-rBE-port-mdpi-v25

46

Pour finir… Exemple d'application multilingue

47

- **Exemple Utile** : traduire les chaines d'une application sans la recompiler et sans la développer 2 fois ou plus.
  - Dans ***res/values/strings.xml*** (dossier par défaut), on définit les chaînes de l'application (ici en Anglais), au lieu de les mettre comme des constantes dans le code source.

```
<resources>
    <string name="app_name">FirstApp</string>
    <string name="hi">Hello</string>
    <string name="main_menu">Main Menu</string>
    <string name="config">Settings</string>
</resources>
```

47

Pour finir… Exemple d'application multilingue

48

- Puis, il suffit de faire des copies du contenu de ce fichier dans les dossiers ***values-fr***, ***values-it***, ***values-de***, etc. et de traduire les textes tout en gardant les attributs '**name**'.
- Voici par exemple ***res/values-fr/strings.xml*** :

```
<resources>
    <string name="app_name">FirstApp</string>
    <string name="hi">Salut</string>
    <string name="main_menu">Menu Principal</string>
    <string name="config">Configuration</string>
</resources>
```

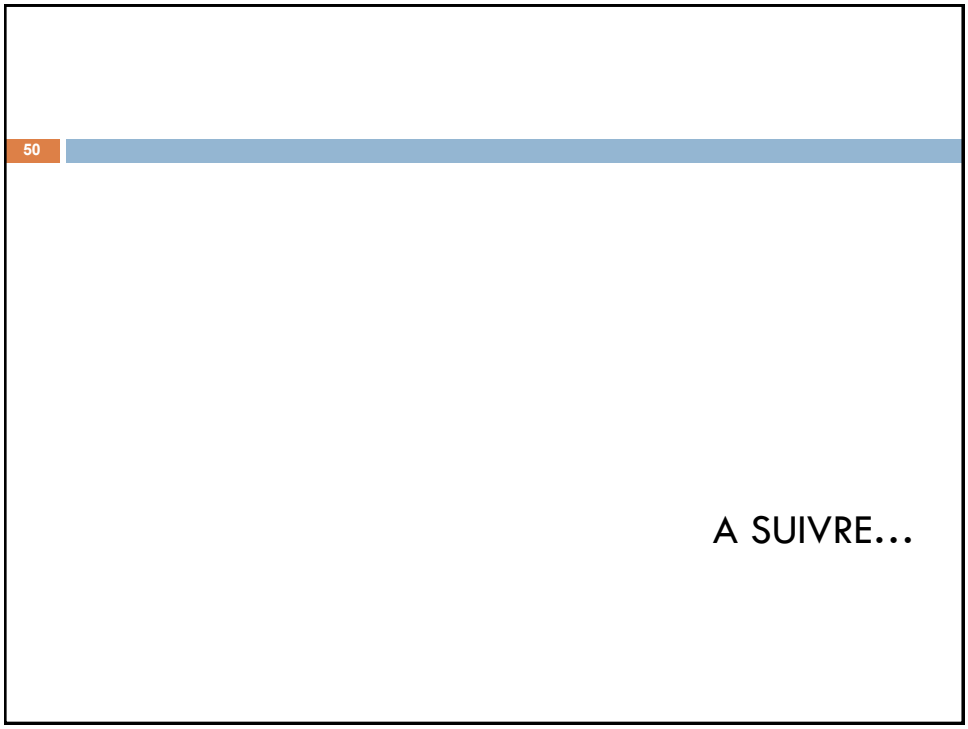
- Le système Android va chercher automatiquement le bon texte en fonction des paramètres linguistiques configurés par l'utilisateur.

48

Récap.

- Android choisit toujours la **ressource la plus adaptée** au téléphone.
- On peut préciser les conditions avec des **qualificateurs** (langue, orientation, densité, taille, version Android).
- Si le téléphone ne trouve pas la ressource exacte, il prend la **plus proche disponible**.
- **Exemple** : une icône ou un texte peut changer selon la langue, la résolution ou l'orientation.

49



50